```
}
static void LFY_device_cleanup(void)
{
unregister_chrdev(161,"LFY_device");
return ;
}
```

As you can see, the above module code contains an include statement for the file LFY_device_header.h. This is a custom header file for our driver. Let me show you that file as well:

```
#ifndef _LFY_DEVICE_H
#define _LFY_DEVICE_H
#include <linux/fs.h>
#include <linux/sched.h>
#include <linux/errno.h>
#include <asm/current.h>
#include <asm/segment.h>
#include <asm/uaccess.h>

char LFY_device_data[80]="Sample data - A Voyage to Kernel";
int LFY_device_open(struct inode *inode,struct file *filp);
int LFY_device_release(struct inode *inode,struct file *filp);
ssize_t LFY_device_read(struct file *filp,char *buffer,size_t count,loff_t *offp );
ssize_t LFY_device_write(struct file *filp,const char *buffer,size_t
count,loff_t *offp );

struct file_operations LFY_ops={
      open: LFY_device_open,
      read: LFY_device_read,
      write: LFY_device_write,
      release:LFY_device_release,
};
int LFY_device_open(struct inode *inode,struct file *filp)
{
        return 0;
}

int LFY_device_release(struct inode *inode,struct file *filp)
{
        return 0;
}
ssize_t LFY_device_read(struct file *filp,char *buffer,size_t count,loff_t *offp )
{
      if (copy_to_user(buffer,LFY_device_data,strlen(LFY_device_data)) != 0 )
        printk( "User-space - copy failed\n" );
      return strlen(LFY_device_data);
}
ssize_t LFY_device_write(struct file *filp,const char *buffer,size_t
count,loff_t *offp )
{
        if ( copy_from_user(LFY_device_data,buffer,count) != 0 )
        printk( "Copy failed\n" );
      return 0;
}
#endif
```

Now, let's analyse the code. One of the important files that the code is referring to is the *linux/fs.h* file. The *file_operations* structure is described in this file. And if you look at this file carefully, you can see that it contains the necessary pointers to functions that we used in our driver. In the case of an actual device, this will help you in performing the basic operations on your device. Here, each field is linked to the address of a function that we used in the driver. The basic operations, like reading, writing, etc, are handled using these.

For your reference, here is the *struct file_operations* code:

```
struct file_operations {
      struct module *owner;
      loff_t (*llseek) (struct file *, loff_t, int);
      ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
      ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
      ssize_t (*aio_read) (struct kiocb *, const struct iovec *, unsigned
long, loff_t);
      ssize_t (*aio_write) (struct kiocb *, const struct iovec *, unsigned
long, loff_t);
      int (*readdir) (struct file *, void *, filldir_t);
      unsigned int (*poll) (struct file *, struct poll_table_struct *);
      int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
      long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
      long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
      int (*mmap) (struct file *, struct vm_area_struct *);
      int (*open) (struct inode *, struct file *);
      int (*flush) (struct file *, fl_owner_t id);
      int (*release) (struct inode *, struct file *);
      int (*fsync) (struct file *, struct dentry *, int datasync);
      int (*aio_fsync) (struct kiocb *, int datasync);
      int (*fasync) (int, struct file *, int);
      int (*lock) (struct file *, int, struct file_lock *);
      ssize_t (*sendpage) (struct file *, struct page *, int, size_t,
loff_t *, int);
      unsigned long (*get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
      int (*check_flags)(int);
      int (*flock) (struct file *, int, struct file_lock *);
      ssize_t (*splice_write)(struct pipe_inode_info *, struct file *,
loff_t *, size_t, unsigned int);
      ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_
info *, size_t, unsigned int);
      int (*setlease)(struct file *, long, struct file_lock **);
};
```

You may also find that many operations are supported by this file. It will even help you to perform many other tasks like reading a directory structure and so on. In our case, we are not performing these types of operations. So you can set the corresponding entries to null. In our code, we used the following format:

```
struct file_operations LFY_ops={
      open: LFY_device_open,
      read: LFY_device_read,
      write: LFY_device_write,
```